"Analysis on rental E-bikes services"

An internship report submitted in partial fulfillment of requirements for Bachelors of Business Administration (Analytics and Big Data)

May – July, 2024

Under the guidance of:

Internal guide

DR. Alok Negi
Assistant Professor
School of Business
UPES,
Dehradun

External Guide

Mr. Saurabh Sharma, DGM, IT
Mr. Manu Bhardwaj, System Analyst
Delhi Metro Rail Corporation
New Delhi

Submitted by

Anushka Saini

Sap Id: 500107311

Enrollment no.:

193222018

BBA (Analytics and

Big Data)

STUDENT DECLARATION

I hereby declare that this submission is my own work and that, to the best of knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of my other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

Anushka Saini

SAP Id: 500107311

Enrollment no.: 193222018

BBA- Analytics and Big data, 2022-2025

School of Business, UPES, Dehradun

ACKNOWLEDGEMENT

We take this opportunity to express our sincere thanks to **Mr. Saurabh Sharma** and **Mr. Manu Bhardwaj** and all other esteemed team of experts at Delhi Metro Rail Corporation for having given me a chance to complete my training. We convey our thanks for their cooperation valuable guidance, constructive criticism, and constant encouragement. We are highly grateful to Metro Corporation for giving us an opportunity for the training and for providing the required facilities. It was a great experience working in the institution and learning from such knowledgeable experts in their respective fields. We hope the knowledge gained here during the training would help us in the development of our career as a business analyst.

CERTIFICATE

This is to certify that the summer internship report entitled “Analysis on rental E-bike services” Submitted by Anushka Saini to UPES for partial fulfillment of requirement foe Bachelors of Business Administration – Analytics and Big Data is a bonafide record of the internship work carried out by her under my supervision and guidance. The content of the report, in full or parts have not been submitted to any other institute or University for the award of any other degree or diploma.

Dr. Alok Negi

Assistant Professor

School of Business,

University of Petroleum and Energy Studies

CIN: U74899 DL 1995 GOI 068150

दूरभाष Tel : 23417910/12
फैक्स Fax : 23417921

दिल्ली मेट्रो रेल कॉर्पोरेशन लि०
DELHI METRO RAIL CORPORATION LTD.

(भारत सरकार एवं दिल्ली सरकार का संयुक्त उपक्रम)
(A JOINT VENTURE OF GOVT. OF INDIA AND GOVT. OF DELHI)

No. DMRC/HR/O&M/Trg/2024 **Dated: 16.07.2024**

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Ms./Mr. Anushka Saini**, student of **BBA (Analytics & Big Data)**, of **University of Petroleum and Energy Studies**, has completed her/his internship in **IT Department** of **DMRC** for a period from **15.05.2024** to **15.07.2024.**

Her/His approach towards the training was satisfied. During this period, she/he was found to be sincere, pro-active and hard-working.

DMRC wishes her/him success in all her/his future endeavours.

16/7/24

(Satyawan Singh)
Manager/HR/O&M

(मेट्रो भवन, फायर ब्रिगेड लेन, बाराखंभा रोड, नई दिल्ली-110 001)
Metro Bhawan, Fire Brigade Lane, Barakhamba Road, New Delhi-110 001

ABSTRACT

Rapid urbanization and population growth in places like Delhi have raised the demand for reliable public transportation. The Delhi Metro, known for its extensive network and simple connections, is relied on daily by millions of passengers, so the travelling for the passengers Delhi metro has introduced rental e-bikes system in collaboration with YULU, it is startup headquartered in Bengaluru founded in 2017. They are providing their services in Bengaluru, Navi Mumbai, Gurugram, Kochi and Delhi. After collaborating with DMRC YULU decided to expand its services from 250 to 25000 through the year span, but as we see of present the e-bike services are not as successful as though. As part of our plan, we developed as model by analyzing all the variables that affects the services by considering some locations on which rental services are available and then devise it by stating the deployment strategy for it to be more utilized.

CONTENTS

Table of Contents

CHAPTER-1

INTRODUCTION

1.1 Background

The idea of establishing mass rapid transit in New Delhi first originated from a 1969 study on the city's traffic and travel patterns. Over the following years, several government departments formed committees to evaluate issues such as technology, route planning, and jurisdiction. In 1984, the Urban Arts Commission recommended a multi-modal transport system, proposing three underground rapid transit corridors and enhancements to the city's suburban railway and road networks.

As one of India's most populous and fast-growing cities, Delhi faces significant transportation challenges. With its expanding population and rising vehicle numbers, the demand for sustainable transit options has become increasingly urgent. This report examines the innovative partnership between Yulu e-bikes and the Delhi Metro Rail Corporation (DMRC) to tackle these challenges by reducing pollution and improving commuter mobility. Yulu, an Indian company headquartered in Bengaluru, provides shared electric two-wheelers in cities such as Bengaluru, Mumbai, Delhi, and others. With 25,000 dockless EVs and over four million users, Yulu is a leader in micro-mobility solutions, offering electric bicycles as an eco-friendly alternative to conventional transport modes.

Fig 1.1 Yulu bikes

However the project didn’t work well in the some parts of Delhi as planned. There were many factors that were affecting the usage of these rental e-bikes. There were either many complaints about the bike or people damaging the bikes.

1.2 Objective

The primary objective for this project was to analyze how to enhance the productivity e- bike and devise a new deployment strategy for the project.

1.3 Literature review

Introduction

The introduction of rental e-bikes by the Delhi Metro Rail Corporation (DMRC) reflects a growing trend in urban mobility solutions aimed at reducing traffic congestion and promoting sustainable transport. This literature review examines various studies, reports, and articles that discuss the implementation, benefits, and challenges of e-bike rental systems in Delhi.

1. Background on E-Bike Adoption

E-bikes have gained popularity globally as an efficient mode of transport, offering a flexible alternative to traditional public transport. Studies indicate that e-bikes can significantly reduce urban congestion and emissions (Petersen et al., 2020). In Delhi, the need for sustainable transport solutions is heightened due to severe air pollution and traffic challenges (Kumar et al., 2021).

2. DMRC's Initiative

DMRC launched its rental e-bike scheme to complement its metro services, aiming to provide last-mile connectivity. According to the DMRC's official reports, this initiative is part of a broader strategy to encourage the use of public transport and decrease reliance on private vehicles (DMRC, 2022).

3. User Acceptance and Behavior

Research indicates mixed user acceptance regarding e-bike rentals. While some studies highlight a positive attitude towards the convenience and environmental benefits of e-bikes (Ghosh & Jain, 2022), others point out concerns about safety, infrastructure, and the initial cost of usage (Sharma et al., 2023). User behavior studies suggest that ease of access, charging infrastructure, and public awareness significantly impact adoption rates.

4. Environmental Impact

Several studies have assessed the environmental impact of integrating e-bikes into urban transport systems. A study by Singh et al. (2022) found that e-bike usage can reduce carbon emissions by up to 30% compared to conventional vehicles for short trips. This aligns with Delhi's goals to combat air pollution and promote sustainable urban living.

5. Infrastructure and Challenges

The success of e-bike rentals hinges on the availability of appropriate infrastructure, such as dedicated bike lanes and charging stations. Research highlights that inadequate cycling infrastructure remains a significant barrier to the widespread adoption of e-bikes (Rai & Thakur, 2023). Additionally, issues related to theft and maintenance of bikes pose challenges that need addressing.

6. Policy Implications

Policy frameworks play a crucial role in the implementation of e-bike rental systems. Studies suggest that supportive policies, including subsidies and incentives for users and providers, can enhance the effectiveness of such initiatives (Kumar et al., 2021). Moreover, integration with existing transport systems is essential for maximizing utility and user convenience.

7. Future Directions

The future of e-bike rentals in Delhi appears promising but requires addressing current challenges. Recommendations include expanding infrastructure, enhancing safety measures, and implementing robust marketing strategies to increase awareness and acceptance among potential users (Verma & Gupta, 2023).

Conclusion

The rental e-bike initiative by DMRC is a critical step toward sustainable urban mobility in Delhi. While there are significant advantages, including environmental benefits and improved connectivity, challenges related to infrastructure and user perception need to be effectively managed. Continued research and adaptive policies will be vital for the success of this initiative.

CHAPTER-2

METHODOLOGY

2.1 Introduction to Random Forest algorithm

Random forest is a supervised learning algorithm. It has two variations – one is used for classification problems and other is used for regression problems. It is one of the most flexible and easy to use algorithm. It creates decision trees on the given data samples, gets prediction from each tree and selects the best solution by means of voting. It is also a pretty good indicator of feature importance.

Random forest algorithm combines multiple decision-trees, resulting in a forest of trees, hence the name Random Forest. In the random forest classifier, the higher the number of trees in the forest results in higher accuracy.

2.2 Random Forest algorithm intuition

Random forest algorithm intuition can be divided into two stages.

In the first stage, we randomly select "k" features out of total m features and build the random forest. In the first stage, we proceed as follows:-

- Randomly select k features from a total of m features where k < m.
- Among the k features, calculate the node d using the best split point.
- Split the node into daughter nodes using the best split.
- Repeat 1 to 3 steps until l number of nodes has been reached.
- Build forest by repeating steps 1 to 4 for n number of times to create n number of trees.

In the second stage, we make predictions using the trained random forest algorithm.

- We take the test features and use the rules of each randomly created decision tree to predict the outcome and store the predicted outcome.
- Then, we calculate the votes for each predicted target.
- Finally, we consider the high voted predicted target as the final prediction from the random forest algorithm

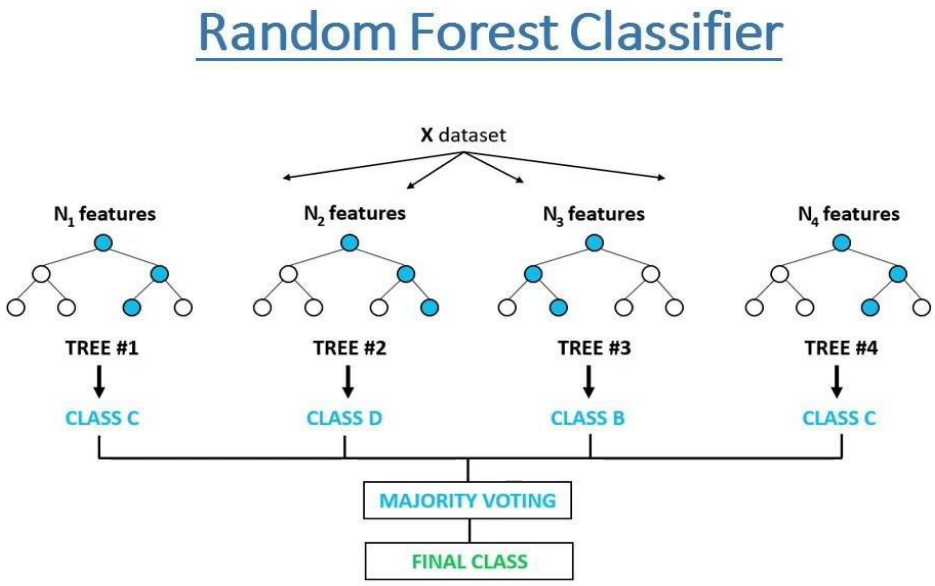


Fig 1.2 Random forest classifier tree

2.1 Advantages and disadvantages of Random Forest algorithm

The advantages of Random forest algorithm are as follows:-

- Random forest algorithm can be used to solve both classification and regression problems.
- It is considered as very accurate and robust model because it uses large number of decision-trees to make predictions.
- A random forest takes the average of all the predictions made by the decision-trees, which cancels out the biases. So, it does not suffer from the over fitting problem.
- Random forest classifier can handle the missing values. There are two ways to handle the missing values. First is to use median values to replace continuous variables and second is to compute the proximity-weighted average of missing values.
- Random forest classifier can be used for feature selection. It means selecting the most important features out of the available features from the training data set.

The disadvantages of Random Forest algorithm are listed below:-

- The biggest disadvantage of random forests is its computational complexity. A random forest is very slow in making predictions because large numbers of decision-trees are used to make predictions. All the trees in the forest have to make a prediction for the same input and then perform voting on it. So, it is a time-consuming process.
- The model is difficult to interpret as compared to a decision-tree, where we can easily make a prediction as compared to a decision-tree.

2.2 Feature selection with Random Forests

Random forests algorithm can be used for feature selection process. This algorithm can be used to rank the importance of variables in a regression or classification problem.

We measure the variable importance in a data set by fitting the random forest algorithm to the data. During the fitting process, the out-of-bag error for each data point is recorded and averaged over the forest.

The importance of the j-th feature was measured after training. The values of the j-th feature were permuted among the training data and the out-of-bag error was again computed on this perturbed data set. The importance score for the j-th feature is computed by averaging the difference in out-of-bag error before and after the permutation over all trees. The score is normalized by the standard deviation of these differences.

Features which produce large values for this score are ranked as more important than features which produce small values. Based on this score, we will choose the most important features and drop the least important ones for model building.

2.3 Difference between Random Forests and Decision Trees

I will compare random forests with decision-trees. Some salient features of comparison are as follows:-

- A random forest is a set of multiple decision-trees.
- Decision-trees are computationally faster as compared to random forests.
- Deep decision-trees may suffer from over fitting. Random forest prevents over fitting by creating trees on random forests

- Random forest is difficult to interpret. But, a decision-tree is easily interpretable and can be converted to rules.

2.4 Programming

The Python programming has been done on the Google colab platform which is a hosted Jupiter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. In the figure below it has shown the necessary libraries have been created for the same. Several factors have been chosen which can possibly affect the usage of the rental e-bikes. These factors are defined as:

YEAR: year 2021 or 2022

HOURS: hour of the day (0 to 23)

SEASON: 1 = winter, 2 = spring, 3 = summer, 4 = autumn

HOLIDAY: if the day was a holiday

WORKING DAY: if the day was a working day (neither a holiday nor a weekend)

TEMPRATURE: temperature in degrees Celsius

ATEMP: sensation of temperature in degrees Celsius

HUMIDITY: relative humidity

AVAILABILITY: total number of rentals in that band

POLLUTION LEVEL: monthly avg of pollution levels

FOOTFALL: ridership per day

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
[ ] import numpy as np
    import pandas as pd
    from matplotlib import pyplot as plt
    from plotly.subplots import make_subplots
    import plotly.express as px
    import plotly.graph_objs as go
    import seaborn as sns
    import missingno as msno
    %matplotlib inline
    import warnings
    warnings.filterwarnings('ignore',category=FutureWarning)
```

```
df = pd.read_csv("/content/rental bike .csv")
df.head()
```

	HOURS	AVAILABILITY	FOOTFALL	SEASON	YEAR	WORKING DAY	HOLIDAY	TEMPRATURE
0	5	12	33296	1	2022	1	0	20.12
1	3	9	8346	2	2021	1	0	34.50
2	6	8	2118	4	2022	1	0	33.50
3	13	8	8256	2	2022	0	0	35.60
4	3	10	5926	4	2021	1	0	34.65

Fig 1.3 rental_bike model (first model)

In the below program the checking of the outliers has been done so that there would not be any irregularities or any missing values in the following to data.

```
[ ] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42 entries, 0 to 41
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   HOURS         42 non-null     int64
 1   AVAILABILITY  42 non-null     int64
 2   FOOTFALL      42 non-null     int64
 3   SEASON        42 non-null     int64
 4   YEAR          42 non-null     int64
 5   WORKING DAY   42 non-null     int64
 6   HOLIDAY       42 non-null     int64
 7   TEMPRATURE    42 non-null     float64
dtypes: float64(1), int64(7)
memory usage: 2.8 KB
```

```
[ ] df.describe()
```

	HOURS	AVAILABILITY	FOOTFALL	SEASON	YEAR	WORKING DAY	HOLIDAY	TEMPRATURE
count	42.000000	42.000000	42.000000	42.000000	42.000000	42.000000	42.000000	42.000000
mean	11.976190	17.357143	21921.904762	2.642857	2021.476190	0.690476	0.071429	32.092857
std	5.421223	6.487592	44873.297110	1.122311	0.505487	0.467901	0.260661	8.216914
min	0.000000	5.000000	2118.000000	1.000000	2021.000000	0.000000	0.000000	17.320000
25%	9.250000	11.250000	6083.500000	2.000000	2021.000000	0.000000	0.000000	29.290000
50%	12.000000	20.000000	10256.000000	3.000000	2021.000000	1.000000	0.000000	33.825000
75%	16.000000	22.000000	15971.000000	4.000000	2022.000000	1.000000	0.000000	38.550000
max	21.000000	30.000000	254000.000000	4.000000	2022.000000	1.000000	1.000000	45.200000

Fig 1.4 Checking for the null values and data stats of the program

```
[11] if df.isnull().values.any():
         print(df.isnull().sum())
     else:
         print('No Missing Values')

     No Missing Values
```

Fig 1.5 Checking missing values

The below image is of heat map which is a visualization tool that uses color and size to show patterns, trends, and relationship in data. They can be used to quickly interpret the data, identify important data points and simplify decision making.

Fig 1.6 Heat map for checking the correlation

In this program it has defined the values and value count under the season feature which we can see below:

```
df['SEASON'] = df['SEASON'].map({1:'winter',2:'spring',3:'summer',4:'autumn'})
df['SEASON'].value_counts()

SEASON
autumn    12
summer    12
winter     9
spring     9
Name: count, dtype: int64
```

Fig 1.7 Defining the values and value count

VISUALIZATION OF DATA

Below is the visualization of the data rental e-bike data.

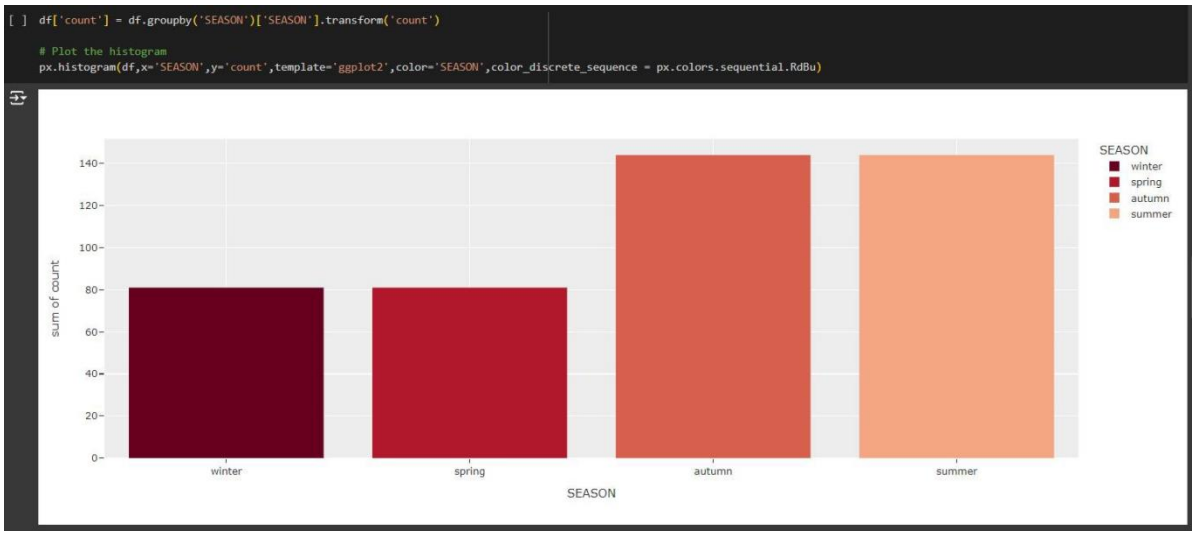


Fig 1.8 Bar plot for count (footfall) in respect to the particular season

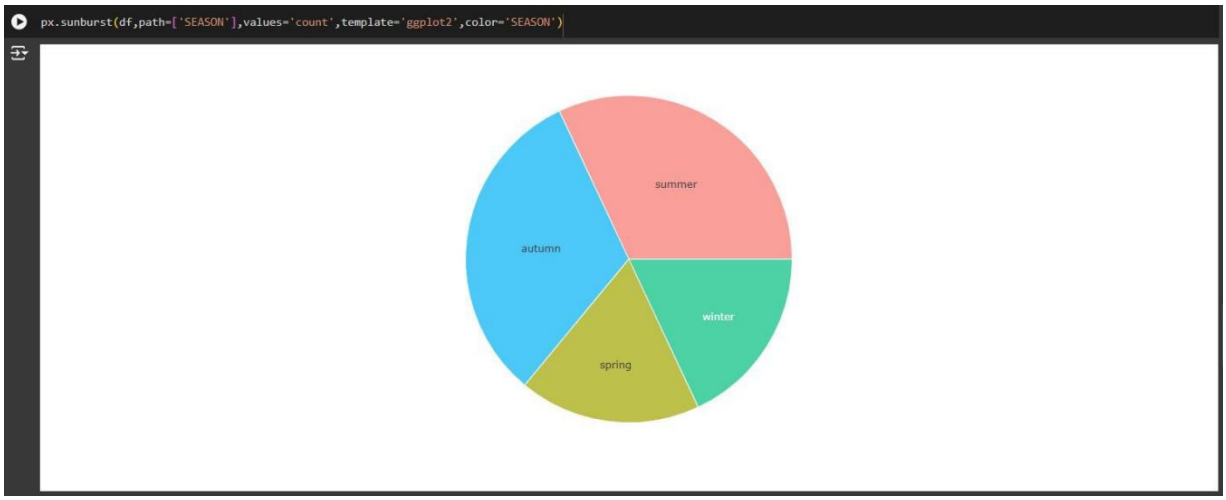


Fig 1.9 Pie chart for count (footfall) in respect to the particular season

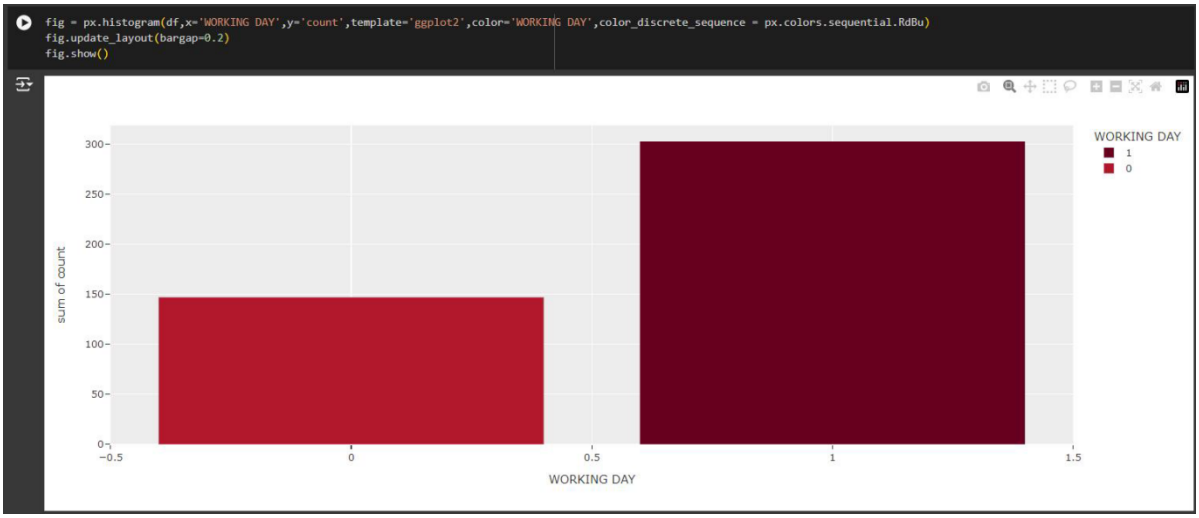


Fig 1.10 Bar plot count (footfall) in respect to the working days in a year

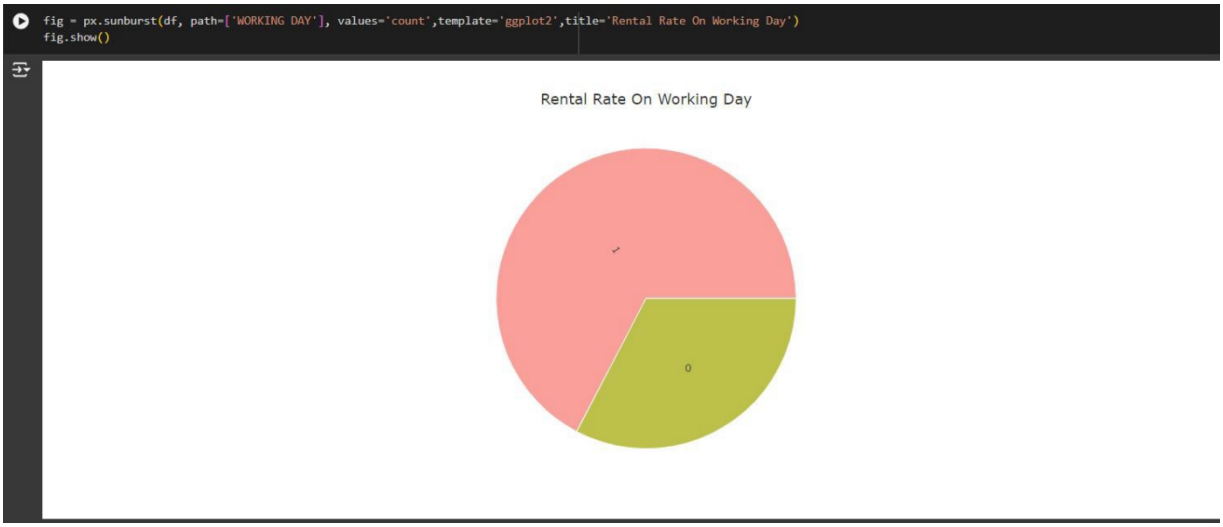


Fig 1.11 Pie chart for count (footfall) on the working day

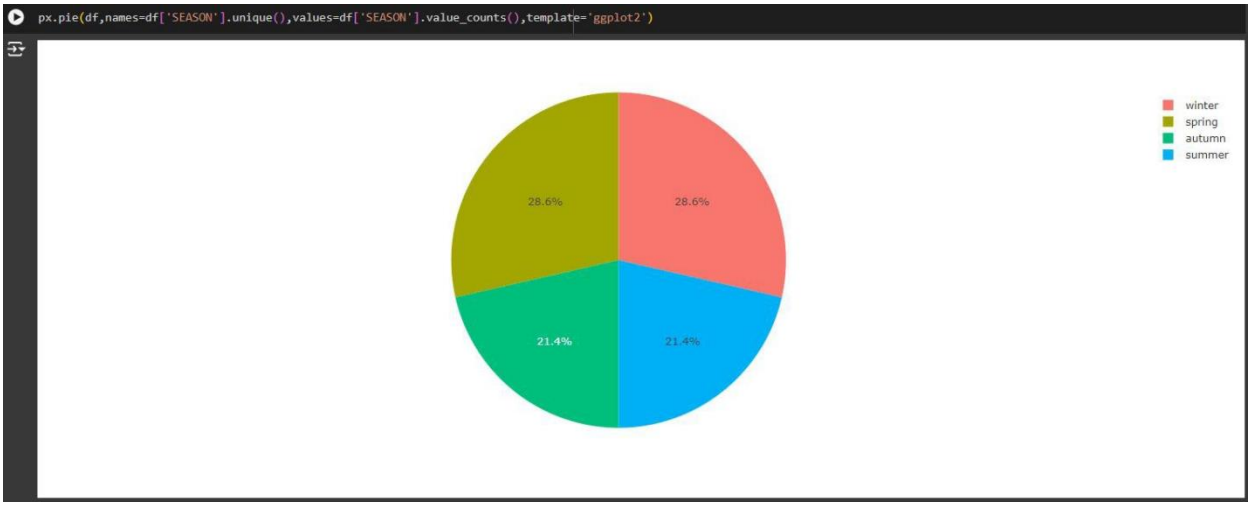
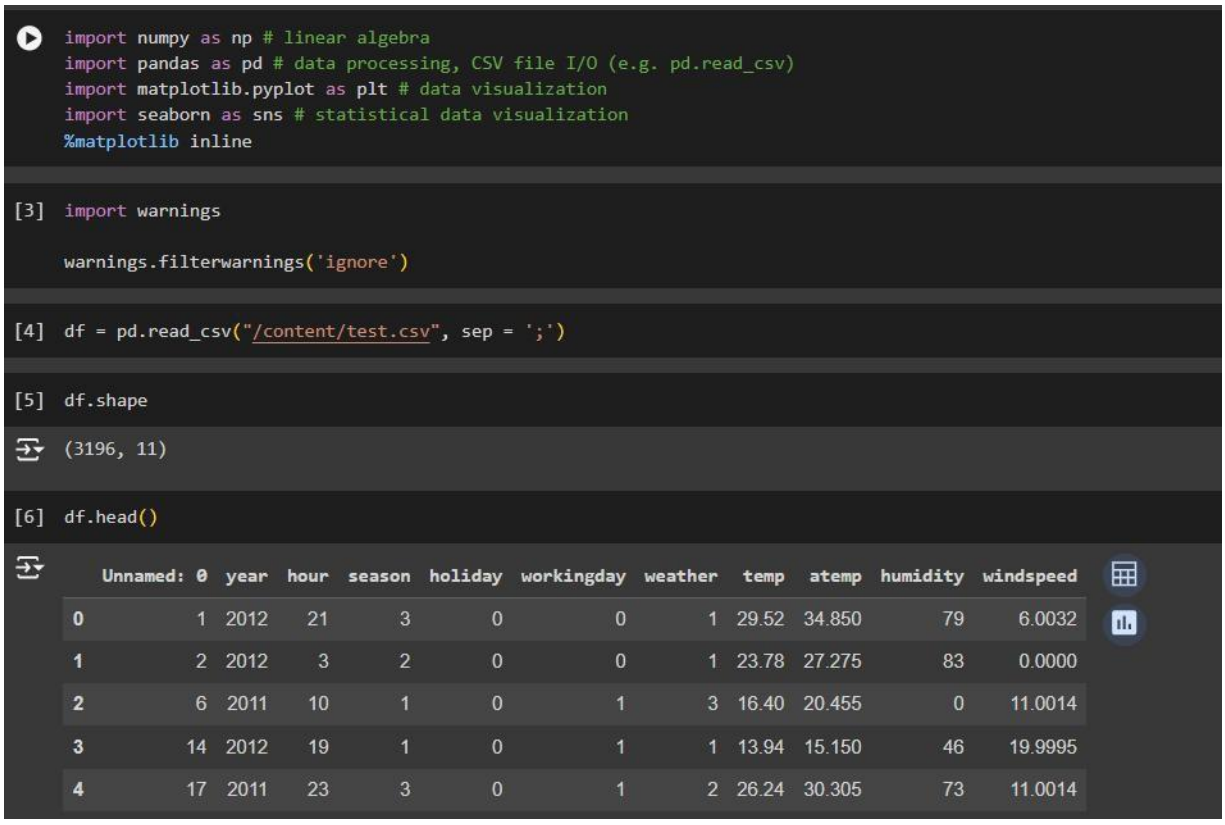


Fig 1.12 Pie chart for count (footfall) in particular season

❖ This is the second data for the rental e-bike for the comparing to see which of these data give the desired results by using random forest classifier.

For this data these are listed feature chosen that affects the usage of the rental e-bike:

- id: time slot identifier (not related to time order)
- year: year (2011 or 2012)
- hour: hour of the day (0 to 23)
- season: 1 = winter, 2 = spring, 3 = summer, 4 = autumn
- holiday: if the day was a holiday
- workingday: if the day was a working day (neither a holiday nor a weekend)
- weather: four categories (1 to 4) ranging from best to worst weather
- temp: temperature in degrees Celsius
- atemp: sensation of temperature in degrees Celsius
- humidity: relative humidity
- windspeed: wind speed (km/h)
- count: total number of rentals in that band



```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt # data visualization
import seaborn as sns # statistical data visualization
%matplotlib inline
```

```
[3] import warnings

    warnings.filterwarnings('ignore')
```

```
[4] df = pd.read_csv("/content/test.csv", sep = ';')
```

```
[5] df.shape
```

```
(3196, 11)
```

```
[6] df.head()
```

	Unnamed: 0	year	hour	season	holiday	workingday	weather	temp	atemp	humidity	windspeed
0	1	2012	21	3	0	0	1	29.52	34.850	79	6.0032
1	2	2012	3	2	0	0	1	23.78	27.275	83	0.0000
2	6	2011	10	1	0	1	3	16.40	20.455	0	11.0014
3	14	2012	19	1	0	1	1	13.94	15.150	46	19.9995
4	17	2011	23	3	0	1	2	26.24	30.305	73	11.0014

Fig 1.13 test.csv model (second model)

```
[7] df.columns

Index(['Unnamed: 0', 'year', 'hour', 'season', 'holiday', 'workingday',
       'weather', 'temp', 'atemp', 'humidity', 'windspeed'],
      dtype='object')

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3196 entries, 0 to 3195
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Unnamed: 0  3196 non-null   int64
 1   year        3196 non-null   int64
 2   hour        3196 non-null   int64
 3   season      3196 non-null   int64
 4   holiday     3196 non-null   int64
 5   workingday  3196 non-null   int64
 6   weather     3196 non-null   int64
 7   temp        3196 non-null   float64
 8   atemp       3196 non-null   float64
 9   humidity    3196 non-null   int64
 10  windspeed   3196 non-null   float64
dtypes: float64(3), int64(8)
memory usage: 274.8 KB
```

Fig 1.14 checking for the null values and columns in the dataset

- In this part the model is being train for the analysis and testing part

```
# import category encoders
!pip install category_encoders
import category_encoders as ce

Collecting category_encoders
  Downloading category_encoders-2.6.3-py2.py3-none-any.whl (81 kB)
     ---------------------------------------- 81.9/81.9 kB 2.2 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.10/dist-packages (from category_encoders) (1.25.2)
Requirement already satisfied: scikit-learn>=0.20.0 in /usr/local/lib/python3.10/dist-packages (from category_encoders) (1.2.2)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from category_encoders) (1.11.4)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.10/dist-packages (from category_encoders) (0.14.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.10/dist-packages (from category_encoders) (2.0.3)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.10/dist-packages (from category_encoders) (0.5.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.5->category_encoders) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.5->category_encoders) (2023.4)
Requirement already satisfied: tzdata>=2022.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=1.0.5->category_encoders) (2024.1)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from patsy>=0.5.1->category_encoders) (1.16.0)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.20.0->category_encoders) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.20.0->category_encoders) (3.5.0)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.10/dist-packages (from statsmodels>=0.9.0->category_encoders) (24.1)
Installing collected packages: category_encoders
Successfully installed category_encoders-2.6.3

[18] cols = X_train.columns

    # encode categorical variables with ordinal encoding
    encoder = ce.OrdinalEncoder(cols=cols)  # Use actual column names

    X_train = encoder.fit_transform(X_train)

    X_test = encoder.transform(X_test)
```

Fig 1.15 Installing of the model

```
# check missing values in variables

df.isnull().sum()
```

```
Unnamed: 0    0
year          0
hour          0
season        0
holiday       0
workingday    0
weather       0
temp          0
atemp         0
humidity      0
windspeed     0
dtype: int64
```

```
[12] X = df.drop(['temp'], axis=1)

     y = df['temp']
```

```
[13] from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.33, random_state = 42)
```

```
[14] # check the shape of X_train and X_test

     X_train.shape, X_test.shape
```

```
((2141, 10), (1055, 10))
```

Fig 1.16 Training of the model on the basis of the target feature temperature (temp)

```
[24] # create the classifier with n_estimators = 100
     clf = RandomForestClassifier(n_estimators=100, random_state=0)

     # fit the model to the training set
     # Discretize y_train to make it suitable for classification
     y_train_cat = pd.cut(y_train, bins=[0, 10, 20, 30], labels=[0, 1, 2], include_lowest=True)
     y_train_cat = y_train_cat.cat.add_categories(-1).fillna(-1) # Handle potential NaNs

     clf.fit(X_train, y_train_cat) # Use the discretized target variable for training
```

```
RandomForestClassifier
RandomForestClassifier(random_state=0)
```

```
# view the feature scores

feature_scores = pd.Series(clf.feature_importances_, index=X_train.columns).sort_values(ascending=False)

feature_scores
```

```
atemp         0.350995
season        0.177965
humidity      0.141395
Unnamed: 0    0.106076
hour          0.082953
windspeed     0.076421
weather       0.024897
year          0.017672
workingday    0.017582
holiday       0.004044
dtype: float64
```

Fig 1.17 Handling the size of the data for training the model and checking the feature importance

```
# Creating a seaborn bar plot
sns.barplot(x=feature_scores, y=feature_scores.index)


# Add labels to the graph
plt.xlabel('Feature Importance Score')
plt.ylabel('Features')


# Add title to the graph
plt.title("Visualizing Important Features")


# Visualize the graph
plt.show()
```

Fig 1.18

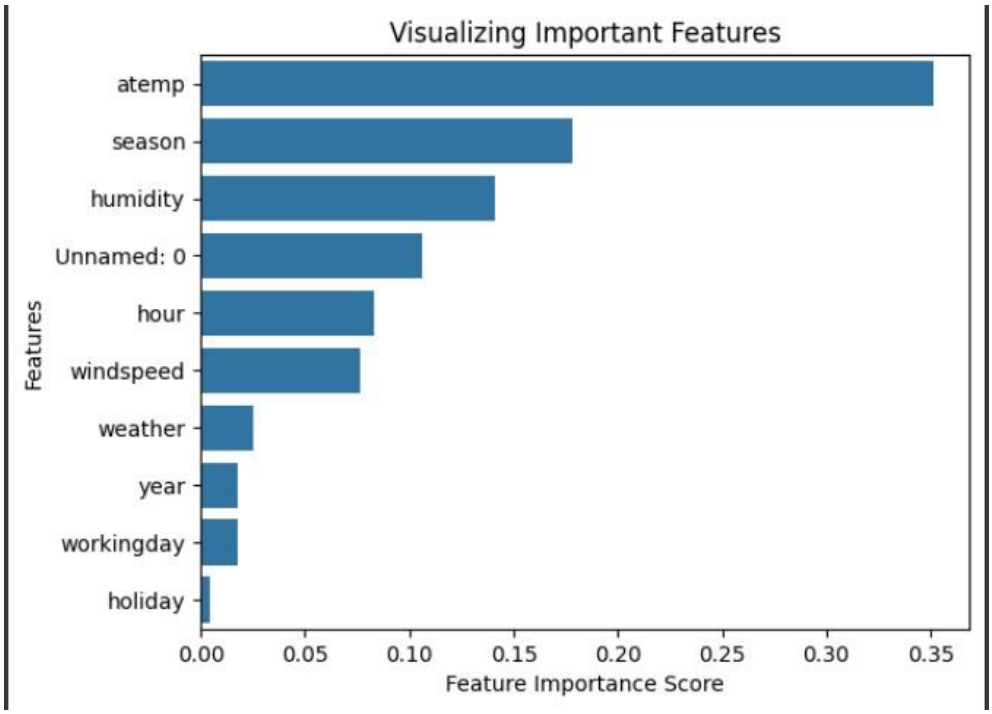


Fig 1.19

Both the figures above shows the feature program and its visualization form of how will each feature affect the target feature i.e. temperature (temp) by which the model is being trained.

```
[21] from sklearn.ensemble import RandomForestClassifier
     from sklearn.metrics import accuracy_score
```

```
# instantiate the classifier

rfc = RandomForestClassifier(random_state=0)



# fit the model
# Include the option include_lowest=True to capture values at the edge of the first bin
y_train_cat = pd.cut(y_train, bins=[0, 10, 20, 30], labels=[0, 1, 2], include_lowest=True)

# Handle potential NaNs (if any remain after including the lowest value)
y_train_cat = y_train_cat.cat.add_categories(-1).fillna(-1) # Replace NaN with a new category

rfc.fit(X_train, y_train_cat)



# Predict the Test set results
y_pred = rfc.predict(X_test)

# Discretize y_test to match y_pred
y_test_cat = pd.cut(y_test, bins=[0, 10, 20, 30], labels=[0, 1, 2], include_lowest=True)
y_test_cat = y_test_cat.cat.add_categories(-1).fillna(-1) # Handle NaNs if any

# Check accuracy score
print('Model accuracy score with 10 decision-trees : {0:0.4f}'. format(accuracy_score(y_test_cat, y_pred))) # Compare discretized y_test
```

```
Model accuracy score with 10 decision-trees : 0.8512
```

Fig 1.20

The above figure shows the accuracy of the model trained that is 0.8512 which is considered good for the model.

The next figures what happens if the target variable is changed to holiday and train the model using it.

```
# declare feature vector and target variable

X = df.drop(['holiday'], axis=1)

y = df['holiday']
```

```
[28] from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.50, random_state = 42)
```

```
[29] # declare feature vector and target variable
     # Remove 'atemp' and 'season' from the list of columns to drop since they are needed by the encoder.
     X = df.drop(['Unnamed: 0'], axis=1)

     y = df['atemp']

     # Instantiate the encoder with the correct columns
     encoder = ce.OrdinalEncoder(cols=['hour', 'season', 'atemp', 'humidity', 'windspeed'])

     # Fit and transform the training data
     X_train = encoder.fit_transform(X_train)

     # Transform the test data
     X_test = encoder.transform(X_test)
```

Fig 1.21

```
# instantiate the classifier with n_estimators = 100

clf = RandomForestClassifier(random_state=0)



# fit the model to the training set

clf.fit(X_train, y_train)


# Predict on the test set results

y_pred = clf.predict(X_test)



# Check accuracy score

print('Model accuracy score with holiday variable removed : {0:0.4f}'. format(accuracy_score(y_test, y_pred)))
```

```
Model accuracy score with holiday variable removed : 0.9725
```

+ Code + T

```
[31] # Print the Confusion Matrix and slice it into four pieces

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print('Confusion matrix\n\n', cm)
```

```
Confusion matrix

 [[1554    2]
 [  42    0]]
```

Fig 1.22

Here the value of accuracy score changes to 0.9725 after changing the target variable which indicates holiday variable will provide more accuracy for prediction than temperature (temp) variable.

```
[32] from sklearn.metrics import classification_report

    print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.97	1.00	0.99	1556
1	0.00	0.00	0.00	42
accuracy			0.97	1598
macro avg	0.49	0.50	0.49	1598
weighted avg	0.95	0.97	0.96	1598

Fig 1.23

Now here in these figures the model training that has been shown is of the first data frame and in this model training the target variable that has been chosen is SEASON which will further affect the model training part.

```
!pip install --upgrade scikit-learn
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (1.2.2)
Collecting scikit-learn
  Downloading scikit_learn-1.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.3 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.3/13.3 MB 53.1 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.25.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.11.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (3.5.0)
Installing collected packages: scikit-learn
  Attempting uninstall: scikit-learn
    Found existing installation: scikit-learn 1.2.2
    Uninstalling scikit-learn-1.2.2:
      Successfully uninstalled scikit-learn-1.2.2
Successfully installed scikit-learn-1.5.0
```

```
rental_bike = pd.read_csv('/content/Book1 .csv')
```

```
X = rental_bike.drop('SEASON', axis=1)
y = rental_bike['SEASON']
```

Fig 1.24

```
# Split the data into training and testing sets (adjust test_size as needed)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# If you need a validation set, split the training set further
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.25, random_state=42)

# Now you have training, validation, and testing sets
print("Training set size:", X_train.shape)
print("Validation set size:", X_val.shape)
print("Testing set size:", X_test.shape)
```

```
Training set size: (23, 10)
Validation set size: (8, 10)
Testing set size: (11, 10)
```

```
rf = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
rf.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

Fig 1.25 The random forest model is being trained

```
# Export the first three decision trees from the forest
for i in range(10):
    tree = rf.estimators_[i]
    dot_rental_bike = export_graphviz(tree,
                                      feature_names=X_train.columns,
                                      filled=True,
                                      max_depth=2,
                                      impurity=False,
                                      proportion=True)
    graph = graphviz.Source(dot_rental_bike)
    display(graph)
```

Fig 1.26 The program for first three decision trees in a random forest program

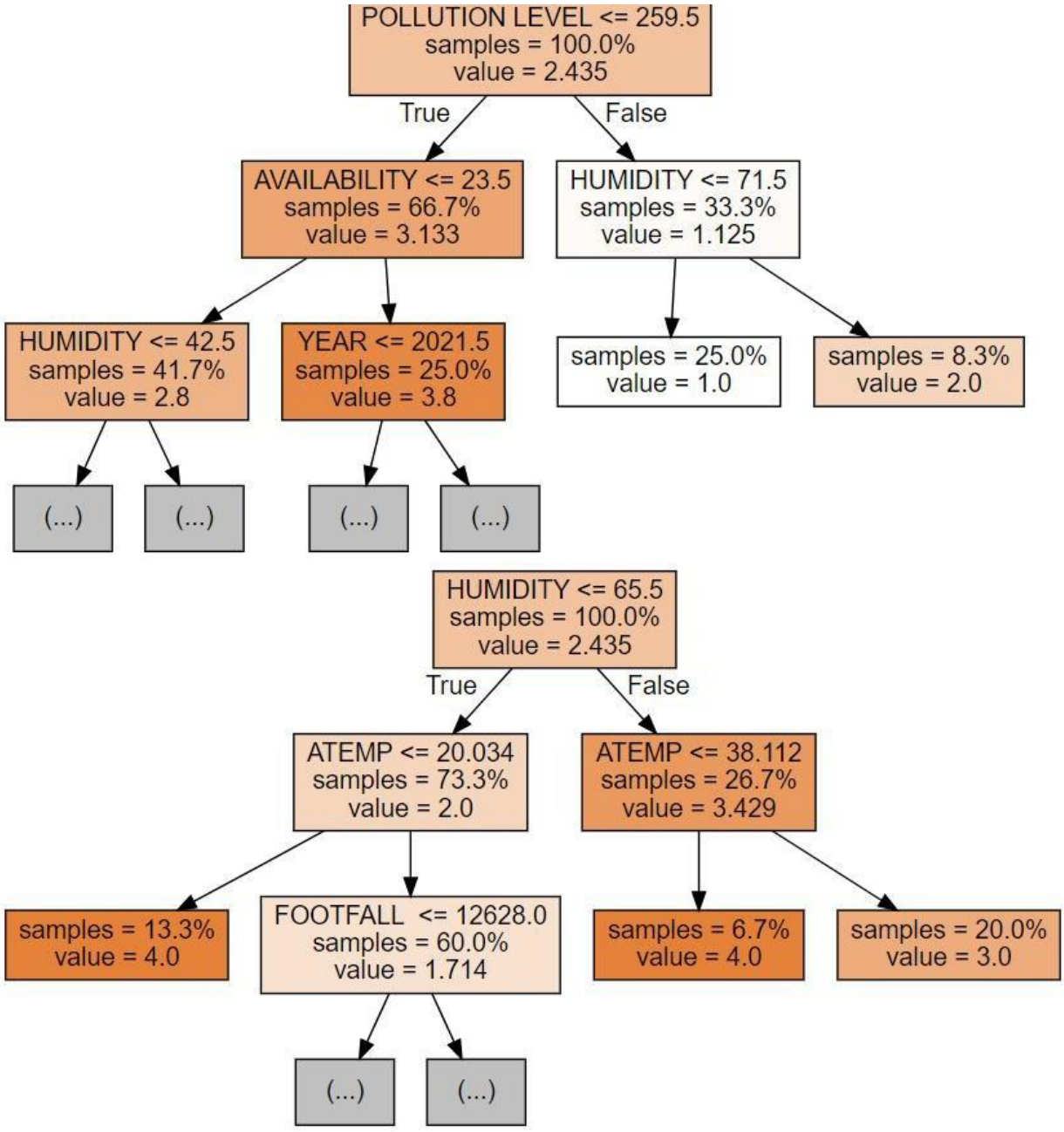


Fig 1.27

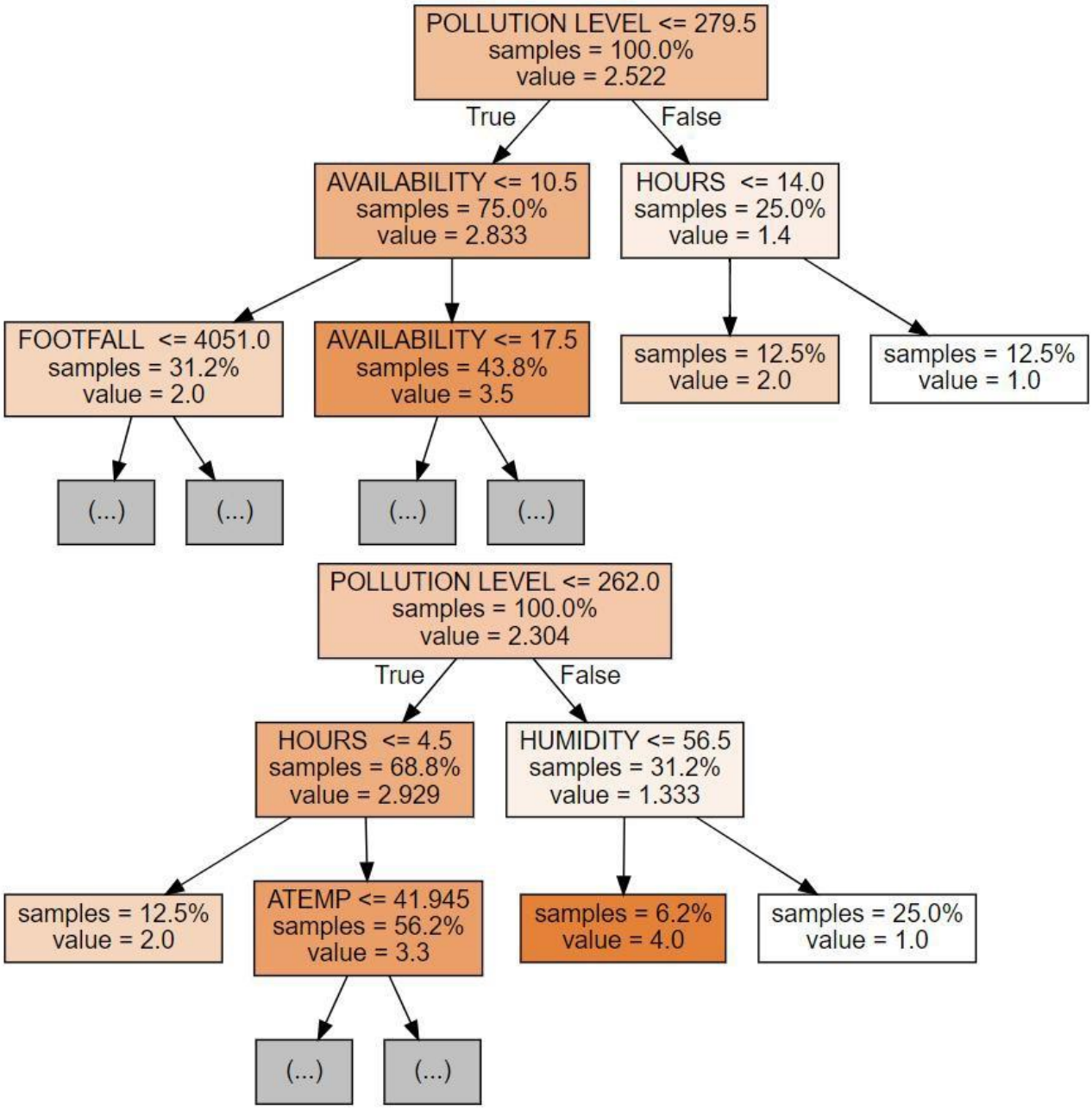

POLLUTION LEVEL <= 279.5
samples = 100.0%
value = 2.522
True
False
AVAILABILITY <= 10.5
samples = 75.0%
value = 2.833
HOURS <= 14.0
samples = 25.0%
value = 1.4
FOOTFALL <= 4051.0
samples = 31.2%
value = 2.0
AVAILABILITY <= 17.5
samples = 43.8%
value = 3.5
samples = 12.5%
value = 2.0
samples = 12.5%
value = 1.0
(...)
(...)
(...)
(...)
POLLUTION LEVEL <= 262.0
samples = 100.0%
value = 2.304
True
False
HOURS <= 4.5
samples = 68.8%
value = 2.929
HUMIDITY <= 56.5
samples = 31.2%
value = 1.333
samples = 12.5%
value = 2.0
ATEMP <= 41.945
samples = 56.2%
value = 3.3
samples = 6.2%
value = 4.0
samples = 25.0%
value = 1.0
(...)
(...)

Fig 1.28

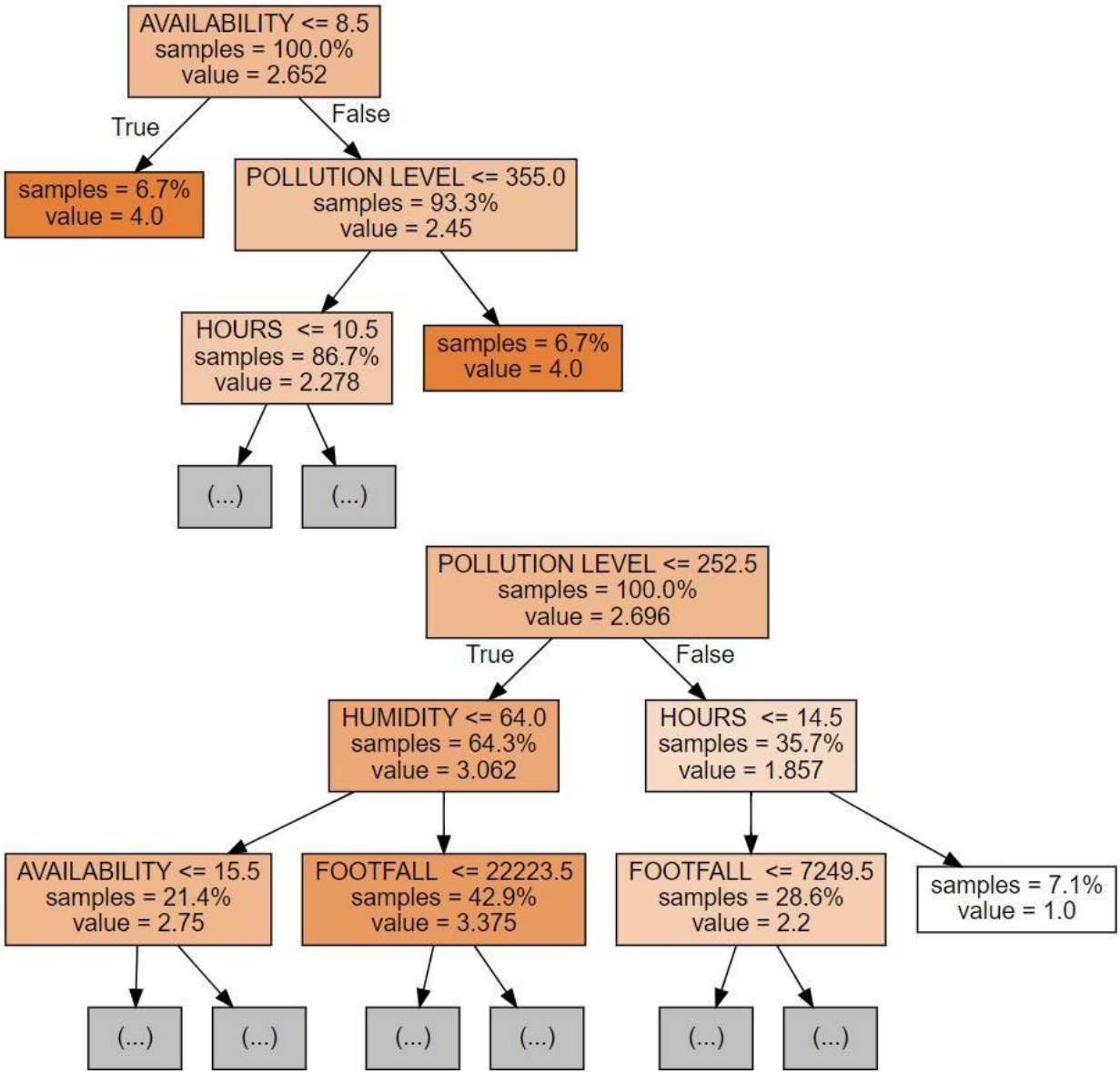

AVAILABILITY <= 8.5
samples = 100.0%
value = 2.652
True
False
samples = 6.7%
value = 4.0
POLLUTION LEVEL <= 355.0
samples = 93.3%
value = 2.45
HOURS <= 10.5
samples = 86.7%
value = 2.278
samples = 6.7%
value = 4.0
(...)
(...)
POLLUTION LEVEL <= 252.5
samples = 100.0%
value = 2.696
True
False
HUMIDITY <= 64.0
samples = 64.3%
value = 3.062
HOURS <= 14.5
samples = 35.7%
value = 1.857
AVAILABILITY <= 15.5
samples = 21.4%
value = 2.75
FOOTFALL <= 22223.5
samples = 42.9%
value = 3.375
FOOTFALL <= 7249.5
samples = 28.6%
value = 2.2
samples = 7.1%
value = 1.0
(...)
(...)
(...)
(...)
(...)
(...)

Fig 1.29

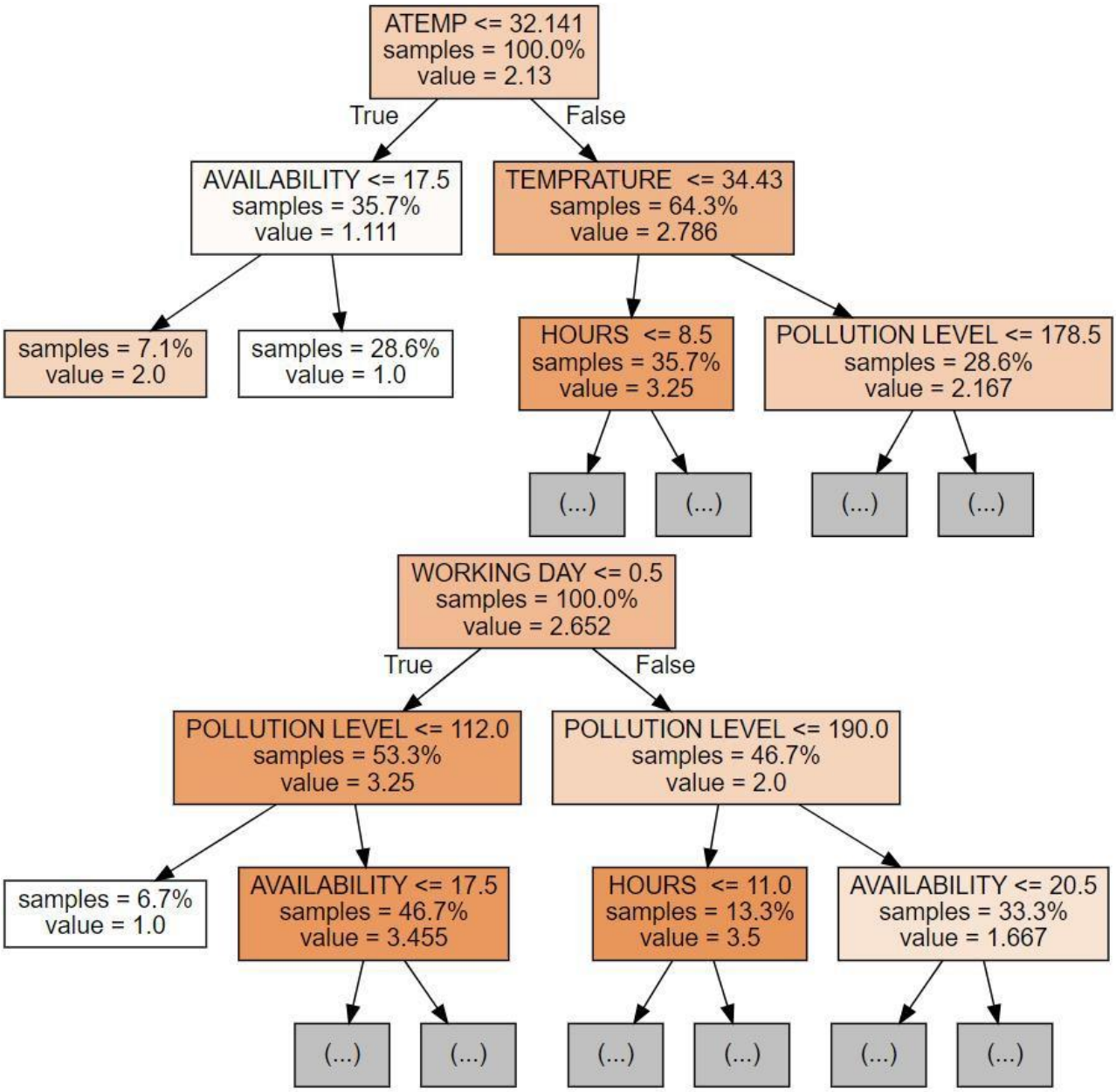

ATEMP <= 32.141
samples = 100.0%
value = 2.13
True
False
AVAILABILITY <= 17.5
samples = 35.7%
value = 1.111
TEMPRATURE <= 34.43
samples = 64.3%
value = 2.786
samples = 7.1%
value = 2.0
samples = 28.6%
value = 1.0
HOURS <= 8.5
samples = 35.7%
value = 3.25
POLLUTION LEVEL <= 178.5
samples = 28.6%
value = 2.167
(...)
(...)
(...)
(...)
WORKING DAY <= 0.5
samples = 100.0%
value = 2.652
True
False
POLLUTION LEVEL <= 112.0
samples = 53.3%
value = 3.25
POLLUTION LEVEL <= 190.0
samples = 46.7%
value = 2.0
samples = 6.7%
value = 1.0
AVAILABILITY <= 17.5
samples = 46.7%
value = 3.455
HOURS <= 11.0
samples = 13.3%
value = 3.5
AVAILABILITY <= 20.5
samples = 33.3%
value = 1.667
(...)
(...)
(...)
(...)
(...)
(...)

Fig 1.30

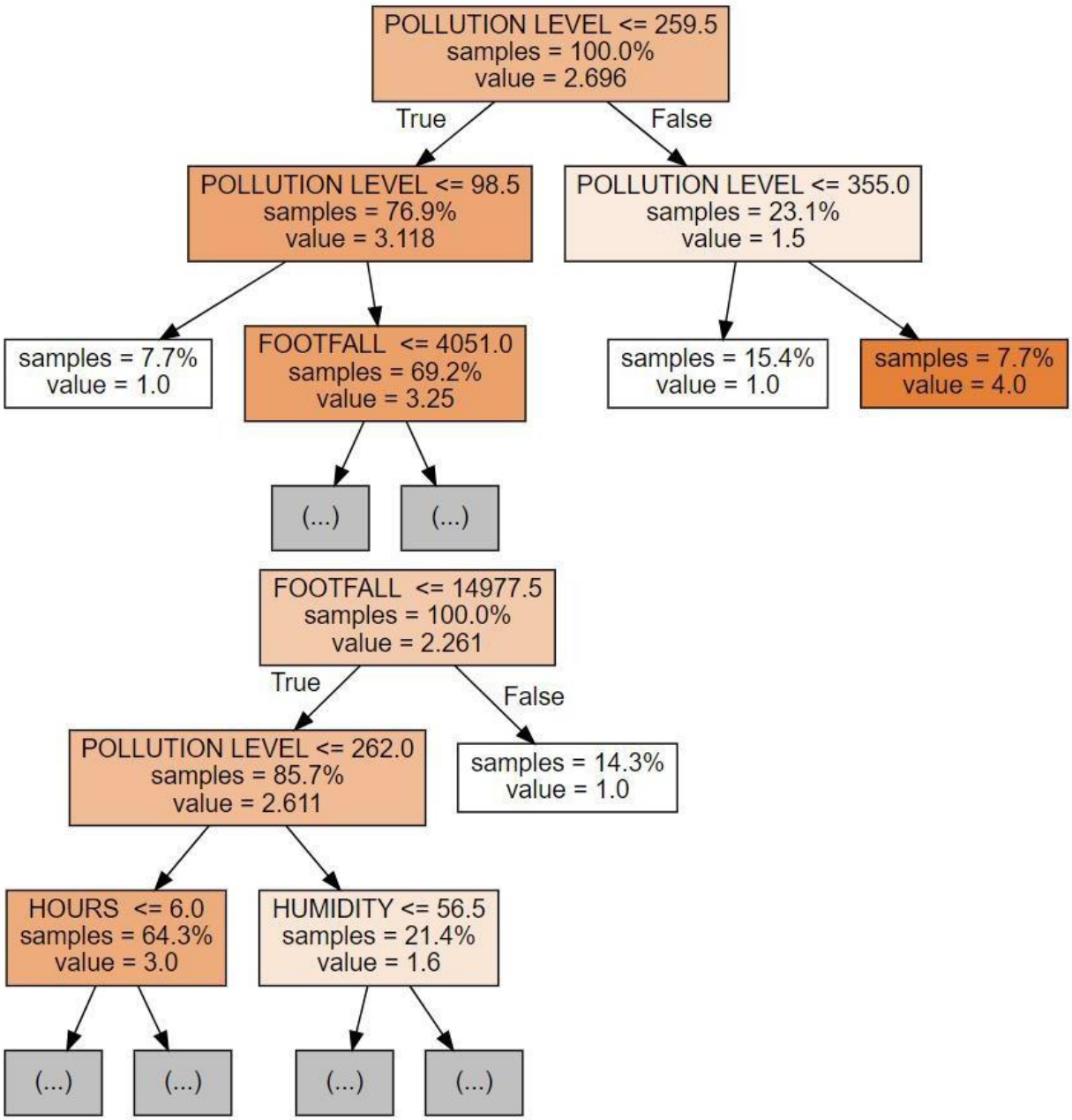


Fig 1.31

Above are the decision trees explaining how the decision in the random forest works.

```
y_pred = rf.predict(X_test)
```

```
importances = rf.feature_importances_
feature_importance = pd.DataFrame({'Feature': X.columns, 'Importance': importances})
feature_importance = feature_importance.sort_values(by='Importance', ascending=False)

print(feature_importance)
```

```
           Feature  Importance
9  POLLUTION LEVEL    0.401130
2         FOOTFALL    0.126902
1     AVAILABILITY    0.115954
8         HUMIDITY    0.109303
7            ATEMP    0.079157
0            HOURS    0.071242
6      TEMPRATURE     0.050154
4      WORKING DAY    0.034096
3             YEAR    0.011429
5          HOLIDAY    0.000633
```

Fig 1.32 Feature importance to the target variable SEASON

```
# instantiate the classifier

rfc = RandomForestClassifier(random_state=42)



# fit the model
# Include the option include_lowest=True to capture values at the edge of the first bin
y_train_cat = pd.cut(y_train, bins=[0, 10, 20, 30], labels=[0, 1, 2], include_lowest=True)

# Handle potential NaNs (if any remain after including the lowest value)
y_train_cat = y_train_cat.cat.add_categories(-1).fillna(-1) # Replace NaN with a new category

rfc.fit(X_train, y_train_cat)



# Predict the Test set results
y_pred = rfc.predict(X_test)

# Discretize y_test to match y_pred
y_test_cat = pd.cut(y_test, bins=[0, 10, 20, 30], labels=[0, 1, 2], include_lowest=True)
y_test_cat = y_test_cat.cat.add_categories(-1).fillna(-1) # Handle NaNs if any

# Check accuracy score
print('Model accuracy score with 10 decision-trees : {0:0.4f}'. format(accuracy_score(y_test_cat, y_pred))) # Compare discretized y_test
```

```
Model accuracy score with 10 decision-trees : 1.0000
```

Fig 1.33

Here the accuracy score of the model is shown is 1.0000 which very good for the model as it will be able to predict the outcomes very accurately.

CHAPTER-3

RESULTS AND DISCUSSION

- Through the analysis.That in Delhi, due to the rise temperature, season and humidity, there is less demand for E Bikes.
- Also concluded that people tend to travel in their private vehicle or by E rickshaw rather than E bikes.
- The bike sometimes stop in the middle of the travelling.
- So the app that is used by the Delhi Metro Yulu bike. Is not user friendly and has various problems, such as

1. Some locations can't be accessed by the bike.
2. A person who wants to use the app can't see if the bike is available at the location.

CHAPTER-4

CONCLUSION AND FUTURE SCOPE

1. In case the company wants to improve the project

To sum up, this project we concluded that to continue the project we should have to add some improvement to the bike.

- Adding a shed to the bike
- Adding a back seat
- Add a mini fan

This is to let people travel in sunny weather & 2 people can travel at the same time.

2. In case the company is looking forward to disinvest in the project

We can introduce a similar model for E- Rickshaw & the benefits it holds

- Available all over Delhi.
- Generate employment for rickshaw driver.
- It hold more people at a time
- It have less investment is the rickshaw is already available in the market

REFERENCES

1. https://www.kaggle.com/datasets/aguado/bike-rental-data-set-uci
2. https://en.wikipedia.org/wiki/Delhi_Metro_Rail_Corporation
3. https://en.wikipedia.org/wiki/Yulu_(transportation_company)
4. https://timesofindia.indiatimes.com/city/delhi/after-gurgaon-delhi-goes-the-eco-friendly- route-with-e-bikes-noida-to-follow/articleshow/72414611.cms
5. https://www.quora.com/What-is-your-review-of-Yulu-a-Bengaluru-based-e-bike-startup